项目一讲解

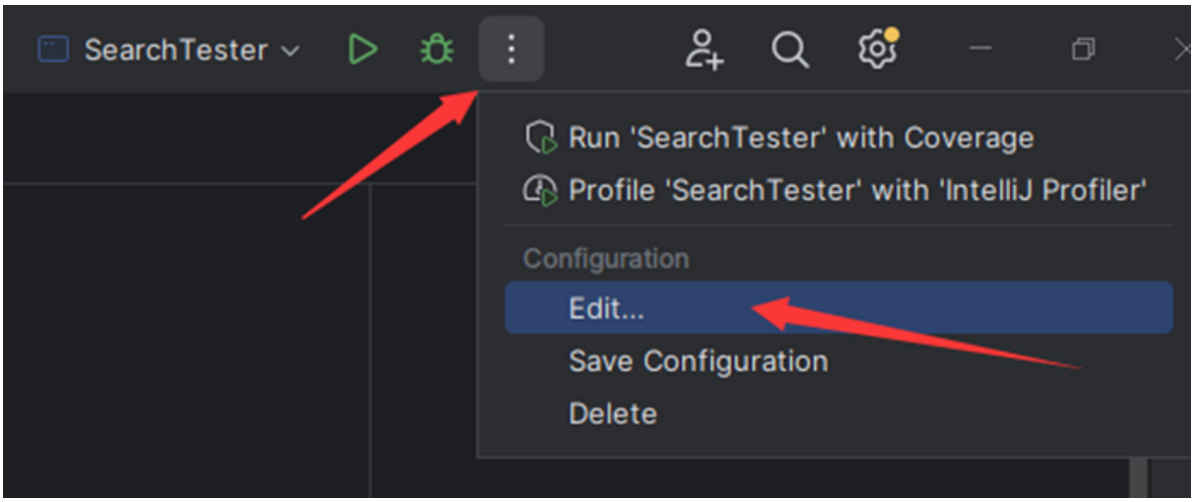
一、程序入口 SearchTester.java

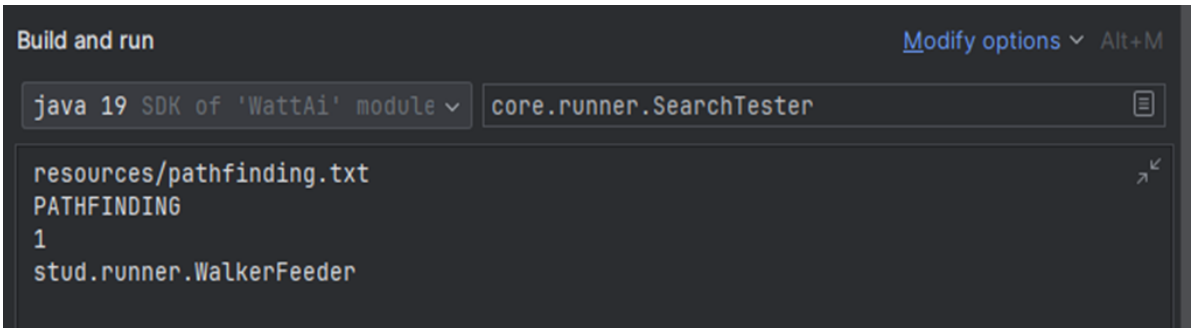
src/core/runner/SearchTest.java

四个参数：

```
/**
 * 对学生的搜索算法进行检测的主程序
 * arg0: 问题输入样例       resources/pathfinding.txt
 * arg1: 问题类型           PATHFINDING
 * arg2: 项目的哪个阶段     1
 * arg3: 各小组的Feeder     stud.runner.WalkerFeeder
 */
```

IDEA中参数的修改：





SearchTest.java的作用是作为主程序，读取问题类型和实例等参数，调用方法进行求解，最后输出结果。

二、EngineFeeder

基类，主要的作用是为搜索算法提供各种素材。

包括：问题实例列表、使用的Frontier、使用的启发函数Predictor。

```
public abstract ArrayList<Problem> getProblems(ArrayList<String> problemLines);
```

```
public abstract Frontier getFrontier(EvaluationType type);
```

```
public abstract Predictor getPredictor(HeuristicType type);
```

返回对应的算法引擎：

```
public final AbstractSearcher getIdaStar(HeuristicType type) {
```

```
public final AbstractSearcher getAStar(HeuristicType type) {
```

```
public final AbstractSearcher getDijkstra() {
```

框架给出了WalkerFeeder.java，是寻路问题的EngineFeeder。

需要参考其编写用于NPuzzle的Feeder。

三、Searcher

AbstractSearcher 抽象基类

进行搜索、生成搜索路径

框架中给出了BestFirstSearcher.java，可以进行参考。

需要根据问题和启发函数，继承AbstractSearcher类，编写用于Npuzzle的Searcher。

四、关于问题的状态

Problem.java： 所有问题的抽象超类

定义了一些与问题相关的变量和函数。

需要编写用于Npuzzle问题的Problem的子类。

```
no usages
public class NPuzzleProblem extends Problem {
    no usages
    public NPuzzleProblem(State initialState, State goal) { super(initialState, goal); }
```

state.java : 问题模型的状态

```java
/**
 * 当前状态下，采用某个action而进入的下一个状态
 *
 * @param action 当前状态下，某个可行的action
 * @return 后继状态
 */
1 usage  2 implementations
public abstract State next(Action action);
```

```java
/**
 * 当前状态下所有可能的Action，但不一定都可行
 *
 * @return 所有可能的Action的List
 */
1 usage  2 implementations
public abstract Iterable<? extends Action> actions();
```

需要编写用于Npuzzle问题的state子类

五、Frontier

用来存储Node的队列，

框架示例给出了ListFrontier，参照编写自己的Frontier类。

问题流程

1、获取Feeder对象

2、从文件读入输入样例文本，feeder获取问题的所有实例

3、获取问题类型和任务阶段，并通过问题类型和任务阶段获取启发函数

4、根据不同启发函数创建searcher，并通过feeder返回到solveProblems()函数中。

寻路问题样例

代码给出了一个寻路问题的可运行的样例

需进行如下修改，将src/core/runner/EngineFeeder.java中第55行改为：

```
54      //生成IdAStar引擎（算法实例）
55      return new IdAStar(frontier);
```

不相交模式数据库

曼哈顿距离

将问题简化，忽略滑块之间的相互作用，将所有滑块到其目标位置的移动次数和作为启发值。这个值是实际移动次数的下界。

问题：没有考虑滑块之间的相互关系。

不相交模式数据库

考虑滑块之间的相互关系，将滑块划分成不相交的组，预先计算每组中的滑块移动到其目标位置所需的最小移动次数，将这样的一组表称为不相交数据库。对于每组中的滑块，使用这些滑块的位置来计算相应表中的索引，检索将该组滑块移动到目标位置所需要的移动次数，将不同组的移动次数相加作为整体启发值。

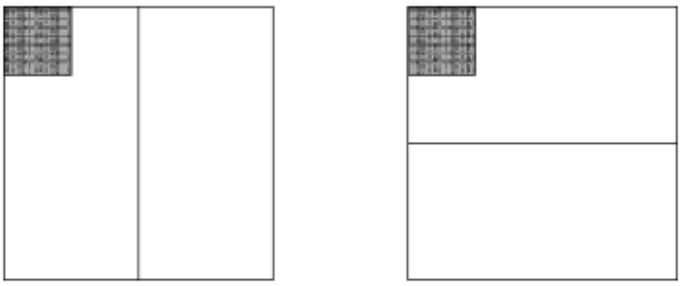


Fig. 4. Disjoint database for Fifteen Puzzle and its reflection.

构建数据库思路示例：

使用广搜，从每组的目标状态开始，遍历每个滑块可能在的位置，并将当前棋盘状态和所需移动的次数保存下来。

相关资料：

Disjoint pattern database heuristics　　(不相交模式数据库原论文)

https://blog.csdn.net/wood_du/article/details/89666736

https://gitee.com/Airsku/npuzzle/tree/master#heuristic-function

……

Zobrist哈希

Zobrist 哈希是一种专门针对棋类游戏而提出来的编码方式,以其发明者 Albert L.Zobrist 的名字命名。Zobrist 哈希通过一种特殊的置换表,也就是对棋盘上每一位置的各个可能状态赋予一个编码索引值,来实现在极低冲突率的前提下在一个整型数据上对棋盘进行编码。其**编码步骤**描述如下:

1) 将棋盘分为最小单位(如果将9X9围棋盘分为81个交叉点),求出每个单位上不同状态数(如围棋盘上的 1 个交叉点有 3 个状态)。

2) 为每个单位上的每种状态生成一个一定范围内(如64位整数)随机数。

3) 对于特定的棋局,将每个单位上的状态对应的随机数作异或运算,所得即为哈希值。

对于Npuzzle问题，以4×4棋盘举例，有4×4个单位，每个单位有16种取值状态。

相关资料：

https://zhuanlan.zhihu.com/p/605462944

https://blog.csdn.net/Wood_Du/article/details/88730885